

MariaDB and Basics SQL

DATABASES — Lezione 2

Relational DBMS - Main Options

- MariaDB
- PostgreSQL
- SQLite3

We will stick to MariaDB, since it is simpler to manage.

Differences

administrator	commands to view and manage structure (listing tables, viewing table schema, displaying and creating users, managing access and permissions, ...)
user	SQLite3 typically usable for small/medium systems; for larger installations use either MariaDB or PostgreSQL
you	learning: no difference, topics apply to all systems

DBMS High-level Architecture

Most of the DBMS systems are based on a client-server architecture:

- There is a server, which manages the databases (DBMS)
- Clients connect to interact with the databases

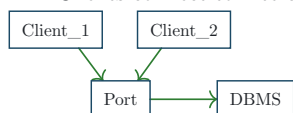


Fig. 1: DBMS client-server architecture

SQLite3 is a notable exception

- SQLite3 is an exception, since it is a library providing structured access to a file (e.g., sqlite3, dbeaver)
- Good fallback option to get started

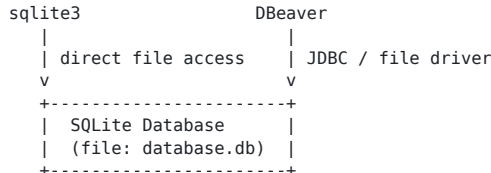


Fig. 1: SQLite3 accesses the database file directly or via a driver

Remarks

- On small setups, client and server run on the same computer
- On Linux-based systems, connections happen via port or socket
- Default ports: 3306 (MariaDB), 5432 (PostgreSQL)

In practice

Installing a DBMS requires to:

- Install the server (which usually comes with a CLI client)
- Install any other client you might want (e.g. DBeaver, php-MyAdmin)
- starting/enabling the server (where required)

Installing MariaDB

- Installing the server and a CLI client (MariaDB getting-started guide)
- Installing a graphical client (DBeaver)
- Other options (Arch Wiki: list of applications — database tools; MySQL — graphical tools)

Fallback option

DBeaver with SQLite3.

Specific Installation Guides: Linux

ArchLinux

```
# Install package
sudo pacman -S mariadb
```

```
# Configure system
mariadb-install-db --user=mysql --basedir=/usr \
  --datadir=/var/lib
```

```
# Start server (and make sure it starts after reboot)
systemctl start mariadb
systemctl enable mariadb
```

```
# Secure installation
mariadb-secure-installation
```

Specific Installation Guides: OSX

Installation on OSX requires homebrew (or docker)

- Installing Homebrew
- MariaDB Formula

```
/bin/bash -c "$(curl -fsSL https://raw.githubusercontent.com/
Homebrew/brew-install mariadb
# mariadb-secure-installation
brew services start mariadb
```

DA VERIFICARE comando troncato nell'originale: l'URL della slide termina con «.../Home»; il resto è tagliato dalla slide.

Specific Installation Guides: Windows

- Instructions here (MariaDB installing guide)
- Graphical installer

Specific Installation Guides: Docker/Podman

- Prerequisite: docker or podman
- Containers are volatile and you can start from scratch every time you want, by launching the command above
- If you start a stopped container or persist data with a volume, nothing will be lost

```
podman pull docker.io/library/mariadb
```

```
podman run --name mariadb \
  --env MARIADB_ROOT_PASSWORD=1234567 \
  --replace \
  --detach \
  -p [IP_ADDRESS]:3306:3306 \
  mariadb:latest
```

Persisting data with podman/docker

```
mkdir ./mariadb-storage
```

```
podman run podman --name mariadb \
  --env MARIADB_ROOT_PASSWORD=12345678 \
  --replace \
  --detach \
  --volume ./mariadb-storage:/var/lib/mysql
-p [IP_ADDRESS]:3306:3306 \
  mariadb
```

DA VERIFICARE podman ripetuto («podman run podman») e \ mancante dopo --volume ...:/var/lib/mysql nella slide originale.

Connecting to the server

Client can connect to the server, specifying host, user, port (usually when different from defaults):

```
mariadb -h [IP_ADDRESS] -P 3306 -p -u root
```

Explanation

<code>-h</code>	host
<code>-P</code>	port
<code>-p</code>	ask for a password
<code>-u root</code>	connects as the root user

Part II

Using a DBMS as a «naive» Excel replacement

Databases

- Databases are made of tables
- Tables have columns with types

- Tables are populated with data
- Fine-grained access control allows to specify which user can do on which databases and tables

Name	Surname	Phone	...
John	Doe	+1...	
Jane	Doe	+2...	

Creating a database and a user

```
-- mariadb -h [IP_ADDRESS] -P 3306 -p -u root
```

```
create database contacts;
```

```
create user 'contacts'@'localhost' identified by '12345678';
```

```
grant all privileges on contacts.* to 'contacts'@'%';
```

Creating a Table

```
USE contacts;
-- CAREFUL HERE!
DROP TABLE IF EXISTS contacts;
CREATE TABLE contacts (
  name VARCHAR(50),
  surname VARCHAR(50),
  phone VARCHAR(20),
  country VARCHAR(50),
  age INT,
  signup_date DATE,
  status VARCHAR(20)
);
DESCRIBE contacts;
```

Populating data

- insert: SQL provides an insert command to enter data
- from dumps: given a DB we can save the commands to create in a file it and, then, of course, use the file to create a copy of the database
- from CSV files

Insert Example

```
USE contacts;
```

```
INSERT INTO
  contacts
  (name, surname, phone, country, age, signup_date, status)
VALUES
  ('John', 'Doe', '+1-555-0101', 'USA', 28, '2025-01-15',
  'Active
  ('Emma', 'Smith', '+44-20-7946-0912', 'UK', 34,
  '2025-02-20', '');
```

```
SELECT * from contacts;
```

DA VERIFICARE valori della INSERT troncati a destra dalla slide originale (righe 'Active e '2025-02-20', ').

Loading from a dump

```
use contacts;
source dump.sql;
```

Exercise

- Use the insert command to insert some records
- Use the load command to load a dump

SQL Basics

```
select <columns>
from <table>
where <condition>;
```

Some queries

- name like ...
- age between 30 and 40
- where signup_date >
- where month(signup_date)
- min, max, avg
- ...